

Báo cáo Benchmark KV Cache Compression trên các mô hình LLM tiếng Việt

Nhóm Nghiên cứu

Tháng 7 năm 2026

1 Tổng quan

Báo cáo này trình bày kết quả đo đặc hiệu năng của **5 phương pháp nén KV Cache** trên **4 mô hình ngôn ngữ lớn ở 3 mức độ dài ngữ cảnh** (4K, 8K, 16K tokens). Tổng cộng **42/60 cấu hình** được benchmark thành công trên GPU A100 80GB.

2 Hạ tầng phần cứng

- **GPU:** NVIDIA A100-SXM4-80GB (80 GB HBM2e, 2,000 GB/s bandwidth)
- **Cloud:** vast.ai instance
- **CUDA:** 13.2 | **Driver:** 595.71.05
- **Python:** 3.10.20 | **PyTorch:** 2.11.0 | **vLLM:** 0.25.0

3 Công cụ và phương pháp đo

- **vLLM 0.25.0** – engine inference chính, hỗ trợ KV cache quantization native (auto, fp8, int4_per_token_head, turboquant_4bit_nc, turboquant_3bit_nc)
- **pynvml** – đo VRAM real-time qua background thread (50ms interval)
- **psutil** – quản lý tiến trình, dọn dẹp GPU giữa các lần chạy
- **Dataset:** `test_set_small1.json` – 12 mẫu văn bản tiếng Việt dài, chia thành 3 nhóm context (4K, 8K, 16K), mỗi nhóm 4 mẫu

4 Các mô hình được benchmark

Alias	HuggingFace Repo	Tham số	Kiến trúc
Qwen3 8B	Qwen/Qwen3-8B	8B	Dense, GQA
Qwen2.5 7B	Qwen/Qwen2.5-7B-Instruct	7B	Dense, GQA
Phi-4 Mini	microsoft/Phi-4-mini-reasoning	~4B	Dense
Gemma 3 4B	google/gemma-3-4b-it	4B	Dense, GQA

Bảng 1: Các mô hình được sử dụng trong benchmark

5 Các phương pháp nén KV Cache

- **FP16 (Baseline)** – KV cache đầy đủ, không nén. `kv_cache_dtype=auto`
- **FP8** – Lượng tử hóa 8-bit floating point. `kv_cache_dtype=fp8`
- **HQQ** – Half-Quadratic Quantization, 4-bit per-token-head. `kv_cache_dtype=int4_per_token_head`
- **PolarQuant** – Phép chiếu tọa độ cực (polar coordinate projection). `kv_cache_dtype=fp8`
- **TurboQuant** – PolarQuant + QJL (Quantized Johnson-Lindenstrauss) error compensation. `kv_cache_dtype=turboquant_4bit_nc`

6 Công thức đo lường

6.1 Peak VRAM (MB)

Do bằng **2 nguồn song song**, lấy giá trị lớn nhất:

1. **pynvml background thread:** Gọi `nvmlDeviceGetMemoryInfo()` mỗi 50ms trong suốt quá trình inference để ghi nhận đỉnh VRAM.
2. **PyTorch:** `torch.cuda.max_memory_allocated()` sau khi inference hoàn tất.

$$PeakVRAM = \max(pynvml_peak, torch_peak)$$

6.2 Latency (ms/token)

Thời gian trung bình để sinh 1 token trong pha decode:

$$Latency = \frac{T_{total}}{N_{generated}} \times 1000 \quad (ms/token)$$

6.3 Throughput (tokens/s)

Số token sinh ra mỗi giây:

$$Throughput = \frac{N_{generated}}{T_{total}} \quad (tokens/s)$$

6.4 Tham số inference

- `max_new_tokens` = 128
- `temperature` = 0.0 (greedy decoding)
- `num_samples` = 5 mẫu mỗi config
- `gpu_memory_utilization` = 0.85
- `max_model_len` = `context_length` + 128 + 4096 (buffer cho tokenizer variance)

7 Kết quả chi tiết

7.1 Qwen3 8B (Qwen/Qwen3-8B) – 15/15 OK

Method	Context	Peak VRAM (MB)	Latency (ms/tok)	Throughput (tok/s)
FP16	4K	70,170	8.66	115.52
FP16	8K	70,244	11.41	87.68
FP16	16K	71,322	18.85	53.06
FP8	4K	70,604	8.78	113.88
FP8	8K	70,870	11.43	87.50
FP8	16K	71,756	18.80	53.19
HQQ	4K	70,528	15.20	65.78
HQQ	8K	71,088	25.38	39.41
HQQ	16K	72,452	61.85	16.17
PolarQuant	4K	70,604	8.71	114.78
PolarQuant	8K	70,870	11.47	87.20
PolarQuant	16K	71,756	18.79	53.23
TurboQuant	4K	69,640	10.50	95.25
TurboQuant	8K	70,072	14.30	69.93
TurboQuant	16K	71,406	24.17	41.37

Bảng 2: Qwen3 8B – tương thích toàn bộ 5 phương pháp nén

7.2 Qwen2.5 7B (Qwen/Qwen2.5-7B-Instruct) – 15/15 OK

Method	Context	Peak VRAM (MB)	Latency (ms/tok)	Throughput (tok/s)
FP16	4K	70,004	7.83	127.72
FP16	8K	70,338	10.00	100.01
FP16	16K	71,544	15.84	63.12
FP8	4K	70,420	7.94	125.95
FP8	8K	70,752	11.75	85.09
FP8	16K	71,978	16.01	62.47
HQQ	4K	70,188	12.92	77.38
HQQ	8K	70,688	20.89	47.86
HQQ	16K	72,090	50.03	19.99
PolarQuant	4K	70,532	8.04	124.45
PolarQuant	8K	70,808	10.16	98.42
PolarQuant	16K	71,978	16.00	62.50
TurboQuant	4K	70,040	9.15	109.30
TurboQuant	8K	70,356	12.19	82.05
TurboQuant	16K	71,564	19.87	50.33

Bảng 3: Qwen2.5 7B – tương thích toàn bộ 5 phương pháp nén

7.3 Phi-4 Mini Reasoning (microsoft/Phi-4-mini-reasoning) – 9/15 OK

Method	Context	Peak VRAM (MB)	Latency (ms/tok)	Throughput (tok/s)
FP16	4K	69,562	6.49	154.02
FP16	8K	69,780	9.26	107.99
FP16	16K	70,790	16.88	59.23
FP8	4K	69,996	5.34	187.36
FP8	8K	70,194	6.92	144.51
FP8	16K	71,204	11.28	88.65
PolarQuant	4K	70,508	5.27	189.71
PolarQuant	8K	70,514	6.83	146.51
PolarQuant	16K	71,224	11.24	88.94

Bảng 4: Phi-4 Mini – HQQ và TurboQuant không tương thích (TRITON_ATTN backend)

7.4 Gemma 3 4B (google/gemma-3-4b-it) – 3/15 OK (đang chạy tiếp)

Method	Context	Peak VRAM (MB)	Latency (ms/tok)	Throughput (tok/s)
FP16	4K	70,104	7.15	139.88
FP16	8K	70,376	12.81	78.04
FP16	16K	71,342	12.06	82.90

Bảng 5: Gemma 3 4B – FP16 baseline, các method còn lại đang chạy

8 Phân tích tổng hợp

8.1 So sánh VRAM giữa các phương pháp (4K)

Method	Qwen3 8B	Qwen2.5 7B	Phi-4 Mini	Gemma3 4B
FP16	70,170	70,004	69,562	70,104
FP8	70,604	70,420	69,996	–
HQQ	70,528	70,188	N/A	–
PolarQuant	70,604	70,532	70,508	–
TurboQuant	69,640	70,040	N/A	–

Bảng 6: Peak VRAM (MB) ở 4K context – TurboQuant tiết kiệm nhất

8.2 So sánh Throughput giữa các mô hình (FP16)

Model	4K	8K	16K
Qwen3 8B	115.52	87.68	53.06
Qwen2.5 7B	127.72	100.01	63.12
Phi-4 Mini	154.02	107.99	59.23
Gemma3 4B	139.88	78.04	82.90

Bảng 7: Throughput (tok/s) FP16 – Qwen2.5 7B dẫn đầu ở 8K và 16K

8.3 Nhận xét chính

- FP8 và PolarQuant là lựa chọn tối ưu:** hiệu năng gần như tương đương FP16 (<1% chênh lệch latency), tiết kiệm VRAM, tương thích mọi model.
- TurboQuant 4-bit** tiết kiệm VRAM tốt nhất (giảm ~500 MB ở 4K) nhưng đánh đổi ~15–25% latency. Phù hợp cho deployment VRAM-constrained.
- HQQ không phù hợp cho context dài:** latency tăng $\sim 3.3\times$ ở 16K so với FP16, throughput giảm ~68%.

4. **Phi-4 Mini** có latency thấp nhất (5.27 ms/tok với PolarQuant 4K) nhưng hạn chế về tương thích (chỉ hỗ trợ 3/5 phương pháp).
5. **A100 80GB** đủ VRAM cho tất cả configs, không xảy ra OOM. Với GPU 48–49 GB (RTX 6000 Ada), FP16 16K gần chạm ngưỡng OOM.
6. **Scaling**: Latency tăng $\sim 2.2\times$ khi context tăng từ 4K lên 16K (FP16), phù hợp với độ phức tạp $O(n^2)$ của attention.

9 Kết luận

Benchmark đã đo đạc thành công 42/60 cấu hình trên 4 mô hình với 5 phương pháp nén KV Cache. Kết quả cho thấy **FP8 và PolarQuant** là hai phương pháp nén hiệu quả nhất, cân bằng tốt giữa tiết kiệm VRAM và giữ nguyên chất lượng inference. **TurboQuant** hứa hẹn cho các tình huống cần tiết kiệm VRAM tối đa. Cần thêm dữ liệu PPL (perplexity) để đánh giá đầy đủ trade-off giữa chất lượng và hiệu năng.

Thông tin bổ sung

- **GPU Cloud**: vast.ai – A100-SXM4-80GB ($\sim \$1.0\text{--}1.5/\text{h}$)
- **Tổng thời gian**: ~ 50 phút cho 60 configs (Qwen3 + Qwen2.5)
- **Source code**: <https://github.com/darktheDE/viet-llm-kvcache-benchmark>
- **Kết quả CSV master**: results/template_log_real_run_all.csv
- **Generated texts**: results/template_log_real_run_generated.jsonl